πDVCR — A Personalizable and Interactive Digital VCR

Research Plan, Edward Chang, UCSB

1 Specific Aims

This project is intended to research toward the development of a *Personalizable Interactive Digital VCR* (πDVCR). The salient features of this πDVCR are:

1: Viewing interactivity: A viewer can pause, slow down, replay and fast-forward a live DTV program so the viewer can watch at whatever pace he or she finds comfortable.

2: Content interactivity: A viewer can interact with the content of a DTV program, e.g., manipulating digital objects, interacting with other viewers, and surfing the Internet.

3: Multiple simultaneous streams: A πDVCR can support multiple-channel and multi-stream viewing at the same time.

To support viewing interactivity, we have proposed a receiver-based architecture [2, 6]. A receiver (a digital VCR or set-top box) in this architecture is equipped with a large disk. A viewer can pause a live program to take a break from viewing while the broadcast stream continues arriving and being written to the local disk. The viewer can resume watching the program after the pause with a delay, can replay or can fast-forward the program to get back in synch with the broadcast stream. The viewer can also record programs, filter out unwanted content and produce a customized program (which a tape-based VCR cannot do). Indeed, since we published our architecture paper in 1997 [5], TiVo and ReplayTV have offered digital-VCR products with the above capabilities.

In this πDVCR project, we plan to extend the architecture for supporting multiple interactive streams on one receiver (i.e., one setup box or one digital VCR). My students and I plan to study data placement, caching and prefetching policies for supporting multiple simultaneous high-speed fast-scan streams. We also plan to modify operating system kernels for supporting real-time scheduling so that continuous media data can be played back without disruption. This architecture gives a receiver a client/server dual role. On the one hand, it acts as a client for broadcasters (or servers) to enable interactivity. On the other hand, it caches streams for servicing a number of end-devices to amortize cost. We believe that this architecture is attractive because it is more feasible to deploy than traditional video-on-demand architectures [3, 4, 7, 8, 14], and it is more cost-effective than the single-user digital-VCR model [9]. We envision that a family that has multiple DTV sets can use just one πDVCR for providing multi-device multi-channel interactive service at home. A multi-camera video surveillance system can use just one PC to support multi-clip replay without affecting live monitoring and recording.

To support content interactivity, we plan to investigate the standards for producing interactive DTV programs that were just put forth by the National Institute of Standards (NIST). The interactive TV standards provide standard APIs for building a number of services including network management, content management, user interface and resource management. We plan to comply to the APIs and at the same time explore efficient techniques to make content interaction efficient. For example, when a data object is referenced, it and all its temporally related objects should be found quickly, and once requested, be retrieved efficiently. We will investigate temporal indexing schemes and experiment with various data placement policies to examine if some rules of thumb can work most of the time.

Finally, we deem that it is critical to study data broadcasting issues that can affect a πDVCR's efficiency and effectiveness. Due to the VBR (variable bitrate) nature of compressed videos, a broadcast channel may have substantial slack for transmitting data objects other than videos for supporting interactivity. We will thus also investigate data broadcasting issues including program scheduling and data packetization.

In summary, this research project can be divided into two thrusts: data services and data receivers, and our contributions will be as follows:

1. Data services: We will study the problem of determining what data are more suitable for broadcast, how frequently an atomic data object should be broadcast, and how data should be indexed and packetized for supporting low-latency channel surfing and interactive operations.

2. Data receivers: We will study how a client manages its resources (CPU, memory, and disk) to support low-latency fast-scans without hiccups during the broadcast of programs. Specifically, our deliverables are the following:

- We will develop a buffer management scheme that substantially reduces DRAM requirement for supporting regular playback, pause and slow-motion, and thus makes DTV affordable.
- We plan to study data placement and indexing policies that improve IO resolution and hence make DRAM use for servicing interactive requests low. With a given amount of DRAM, employing our policies can thus support more simultaneous interactions on more TV sets.
- We plan to study caching and prefetching techniques that, on the one hand, satisfy the real-time data requirement of the continuous media data (i.e., video and audio), and on the other hand, minimize the hiccups in responding to an interactive requests.
- We plan to modify Linux process scheduler to support real-time process scheduling.

In the remainder of this proposal, we describe the challenges that the design of a πDVCR faces and our research methods and directions.

2 Relevance

The Federal Communications Commission (FCC) has mandated TV broadcasters to transmit TV in the digital format by the year 2006. The market of interactive DTV has huge potentials. A recent study by a Dataquest report [1] shows that the number of people watching TV while simultaneously surfing the Web has risen from 8 million in 1998 to 27 million in 1999. Many of these "telewebbers" use the Internet to look up more information about a news story or to chat with other viewers and vote in online polls while they are watching a show. The increasing number of telewebbers attests to the potential for including TV-like functionality in the PC and Internet functionality through television and interactivity in TV programs. In addition, according to Forrester Research (http://www.forrester.com/), interactive TV will be worth $20 billion in advertising, commerce and subscription revenue by the year 2004. We believe that the technologies developed by this research project will help enable a wide variety of interactive multimedia applications in education, commercial, government and military sectors.

3 Research Thrusts

In accordance with our main focus, this section is devoted to discussing the challenges and our research methodologies for designing data services and data receivers. We commence this section with an overview on the characteristics of broadcast data. We then show how the design of data services and data receivers takes data characteristics into consideration in Section 3.2 and Section 3.3, respectively.

3.1 Data Characterization

Beyond transmitting video data, digital broadcasting can deliver to clients many different data objects, among them, 3D models, HTML texts, and images. Data objects can be continuous, textual, binary, and so forth.

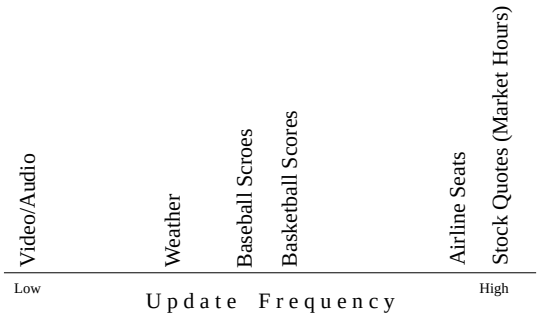


Figure 1: Update Frequency.

These data objects can be characterized using their *size*, *update frequency*, *delay sensitivity*, etc. Data characteristics are considered in determining data scheduling and packetization policies at the server side (Section 3.2), and are considered in managing resources at the client side (Section 3.3).

3.1.1 Size

The size of a data object is measured by the amount of storage space (and thus the transmission bandwidth) the object needs. The size of a data object can vary from a few bytes to several gigabytes. Continuous data (video streams) can be voluminous. For example, a one-hour 19 Mbps HDTV program takes up 8.55-GByte space. Textual data (captions, HTML and XML documents, etc.) in general occupy far less space than continuous data. Other data, such as images and applets, can be very copious or scant.

3.1.2 Update Frequency

We use F_i to represent how frequently the ith data object, D_i, is updated. Static data objects, such as movies and CDs, do not change over time. Once a static object is delivered, the server does not need to worry about the data being out of date. Dynamic data objects, such as stock quotes, sports game scores and the seat availability on a flight, can change over time but can also become static. For instance, once the outcome of the last U.S. Open Tennis Tournament has been decided, it will not change.

Different dynamic data objects have different update frequencies. For instance, a stock quote during daytime may change every few seconds, while the score of a sports game does not change as often. To describe the update behavior of data object D_i, we assign a stochastic process to the object. Stock quotes, for instance, can be modeled as a uniform process. The score of a sports game may observe a Poisson process during the game, and becomes constant once the game is over.

To characterize the *freshness* of an object D_i, we use the *age* τ_i of the object. The age of a static object, such as a movie and a CD, remains zero since it is always fresh. The age of a dynamic object D_i at time t is defined as follows:

$$\tau_i = t - T_{U_i}, \tag{1}$$

where T_{U_i} denotes the time when the object was last updated.

Age is a critical factor to determine the value of an object. An ageless object is always fresh, so its value is constant. The value of a dynamic object often decays with age. For instance, a stock quote with a one-minute delay may not be useful for making real-time trading decisions. Out-of-date seat availability information on a flight may result in double booking. The server should attempt to make the broadcast data as fresh as possible to maximize the usefulness of the data. Figure 1 shows the update frequencies of some example data objects.

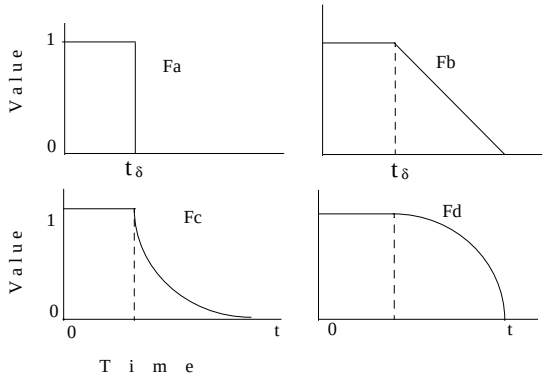


Figure 2: Delay Function Examples.

To quantify the usefulness of a data object, one can specify a value function for it. Figure 2 shows four representative examples. The x-axis in the figures depicts the update delay (age); the y-axis depicts the value of the object normalized to from zero (worthless) to one. Function F_a depicts an object that can tolerate update delay up to a period of time, t_δ, and then becomes worthless. Function F_b shows the value of an object that decays linearly after t_δ. Functions F_c and F_d have other value-decay patterns. Function F_a is a good value function for a stock quote. Function F_b may be a good value function for the score of a baseball game since a slight delay is tolerable. Function F_c may be a good value function for traffic reports. Function F_d, which maintains a high value for a long period of time before rapidly decaying, may be a good value function for weather reports. Without losing generality, we define the value for object D_i as a function of its age: $f_i(\tau)$, where τ denotes the age of the object.

3.1.3 Delay sensitivity

There are two types of delay, *initial delay* and *hiccup*. Initial delay is the time between when an object is requested by a client and when the object is delivered to the client. Hiccup is used to describe the delivery interruption of continuous data objects. For instance, a client waits for five minutes to start receiving a movie that she requested, thus the initial delay for watching the movie is five minutes. If the data cannot be delivered to satisfy the real-time playback requirement of the movie after it is started, every disruption to the playback is counted as one hiccup. Textual data, images and applets do not have real-time constraints if they are processed independently of a continuous media stream. Any data, however, must satisfy real-time requirements if the data must be synchronized with a continuous stream. For instance, the display of the captions of a movie must be in synchronization with the audio track.

Value functions can also be used to characterize the effect of initial delay and hiccup. For instance, we can use value functions similar to those used for data freshness, which describe the value of a data object as time changes, to describe how the value of an object changes as its delay/hiccup increases.

Figure 3 shows the sizes and delay sensitivity of some representative objects. The x-axis in the figure represents the delay sensitivity from high (on the left-hand side) to low (on the right-hand side); the y-axis gives the size of an object. At one extreme, videos are large and have strict real-time constraints. At the other extreme, captions are small and not very sensitive to delay.

Now that we have shown how data can be characterized, we discuss how the design of data services and data receivers takes these characteristics into consideration.

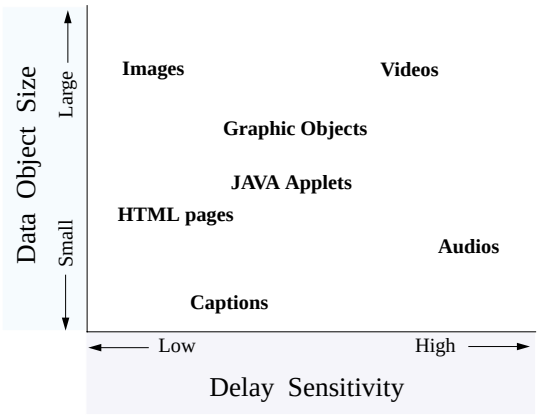


Figure 3: SIze vs. Delay.

3.2 Data Services

A digital channel can transmit both TV programs and conventional data such as Web and binary data because its encoding flexibility. Moreover, because of the VBR (variable bitrate) nature of encoded videos, a broadcast channel has slack for transmitting data objects other than videos. Many challenges exist, however, for designing an effective broadcast model to deliver data to a large number of clients, and two issues we plan to investigate are:

- Should data be placed on the Internet or should it be broadcast? If a data object is to be broadcast, how should its broadcast be scheduled to maximize profit?
- If multiple objects are to be broadcast, how should these objects be packetized so that the delay of delivery is minimum and the channel utilization is maximum?

In the remainder of this section we discuss how we plan to tackle these problems.

3.2.1 Broadcast Scheduling

Given data objects and a time period, a server broadcasts these objects periodically to maximize its profit. How many times each object should be broadcast depends on its size, cost, delay sensitivity, and other characteristics. We can formulate the problem of determining broadcast frequencies of different data objects in the following way.

Assume that a server needs to broadcast M data objects $D_1, \ldots, D_M$ in a time period T. Let the broadcast bandwidth be BB, and the cost of broadcasting a byte be C_{byte}. Let N_i denote the number of times the data object D_i is broadcast during the time T. The scheduling objective function of the server can be modeled as

$$Max \sum_{1}^{M} (R_i - CB_i), \tag{2}$$

where R_i denotes the revenue of broadcasting D_i, and CB_i denotes the cost of broadcasting D_i. We would like to schedule the broadcast in a way that the total profit (revenue minus cost) is maximized.

One way to model the revenue of broadcasting D_i is as follows:

$$R_i = K_i \times f_i(\tau_i) \times g_i(\tau_i) \times F_i. \tag{3}$$

That is, the revenue of broadcasting D_i is characterized by the amount of money each request for the object is worth multiplied by the number of requests F_i. The worth of a request can be measured by the quality of the service, which is represented by $f_i(\tau_i) \times g_i(\tau_i)$, where τ_i represents the average delay caused by the broadcast schedule, i.e., $\tau_i = T/N_i$. We use $f_i(\tau_i)$ to gauge the freshness of the data and $g_i(\tau_i)$ to gauge the effect of the delay on the service. Finally, we multiply the quality with a constant K_i to translate quality into payment.

The cost of broadcasting D_i depends on the usage of the bandwidth, which is the number of bytes transmitted times the transmission cost per byte. We can thus express CB_i as

$$CB_i = C_{byte} \times N_i \times S_i. \tag{4}$$

Finally, the total amount of data broadcast cannot exceed the channel capacity BB. This constraint can be written as

$$\sum_{i}^{M}(N_i \times S_i) \leq BB \times T. \tag{5}$$

Replacing R_i and CB_i in Equation 2 with the right-hand side of Equations 3 and 4, the objective function can be rewritten to the following form:

$$Max\{\sum_{1}^{n}(K_i \times f_i(\tau_i) \times g_i(\tau_i) \times F_i - C_{byte} \times N_i \times S_i)\} \tag{6}$$

We can see simple relationships between parameters by letting $BB = \infty$. (We will install this constraint shortly.) Solving these N_i's requires precise knowledge of each $f_i(\tau_i)$ and $g_i(\tau_i)$, which we may not have. But we know that the value functions $f_i(\tau_i)$ and $g_i(\tau_i)$ are monotonically decreasing. That is, the less frequently a data object is broadcast, the longer the delay to keep the data value up-to-date at the client side, and thereby the less satisfied a user is. Since $f_i(\tau_i)$ and $g_i(\tau_i)$ are both monotonically decreasing with respect of τ_i, R_i has the same property. A representative R_i can thus be written as:

$$R_i = M_i - \frac{F_i \times K_i \times T}{N_i^{\beta_i}}, \tag{7}$$

where M_i is the maximum revenue that the server can attain by broadcasting data D_i infinite number of times and K_i is a constant factor. Since the object can only be broadcast a limited number of times, its revenue is M_i minus a factor $\frac{F_i \times K_i \times T}{N_i^{\beta_i}}$, where β_i is a positive number. The larger β_i is, the faster the revenue decays with update and transmission delay.

To achieve the maximum profit, the first partial derivative of each N_i should be equal to 0. That is,

$$\frac{F_i \times K_i \times T \times \beta_i}{N_i^{\beta_i+1}} - C_{byte} \times S_i = 0. \tag{8}$$

Isolating N_i on the left-hand side of the equation, we obtain

$$N_i = \Big(\frac{F_i \times K_i \times T \times \beta_i}{C_{byte} \times S_i}\Big)^{\frac{1}{\beta_i+1}} \tag{9}$$

Based on Equation 9, we can observe the following relationships:

1. $F_i \times T \uparrow$: If the number of viewers for data object D_i increases, N_i should also increase. Intuitively viewed, if more clients request object D_i, it should be broadcast more often to meet the clients' requests.

2. $C_{byte} \times S_i \uparrow$: As the size of the data D_i or the cost to broadcast increases, N_i should decrease.

3. $\beta_i \uparrow$: The smaller the value of β_i, the more significantly the changes in revenue and cost affect N_i. In other words, if a data object is updated frequently or is delay sensitive, changes in revenue or cost have a more significant effect on the optimal broadcast frequency. On the other hand, if a data object is not delay sensitive, changes in revenue or cost do not affect N_i much.

Now, if we take the bandwidth constraint into consideration, the problem becomes more challenging. Assume we have a limited bandwidth $BB < \infty$, then different data objects will compete for the bandwidth. To determine which data objects should have more frequent broadcast schedules, one way is to consider the marginal profit of broadcasting each object D_i. One can pick the D_i that can achieve the maximum marginal profit by following a greedy heuristic. We will investigate scheduling policies that can compute optimal schedules at minimum cost.

Note that not all data are suitable for broadcasting. If the profit of broadcasting a data object is negative, we may consider just putting the data on the Internet.

3.2.2 Packetization

Suppose M objects are to be broadcast and the ith object has size S_i and broadcast frequency N_i that has been determined. we need to determine how to package these M objects so that the broadcast bandwidth requirement is the minimum. We describe and analyze two schemes in this section: the Carousel scheme and the Pyramid scheme. The Carousel scheme schedules time slots and the Pyramid scheme schedules sub-channel allocation. We plan to develop a variety of schemes based on these two basic schemes.

3.2.2.1 The Carousel Scheme

The Carousel scheme transmits data objects (or DTV programs) in a fixed order in a periodic fashion. How to schedule a given set of periodic tasks to meet their deadlines has been widely studied in the real-time community. An effective scheme to test the schedulability of the tasks is Rate Monotonic Scheduling (RMS) [12, 13]. To formulate the broadcast schedule, we first select the broadcast period for each data object and compute the time it takes to transmit the object. Suppose we have M different data objects denoted as D_1 to D_M. We can compute the period to broadcast D_i, denoted as T_i, and the time it takes to transmit D_i, denoted as E_i, by using the following equations:

$$T_i = T/N_i, \quad E_i = S_i/BB.$$

According to RMS, a set of M objects is packetizable (schedulable) if the following conditions can be met

- All objects are broadcast periodically,
- Objects are not synchronized with one another, and
- An object with a shorter period has a higher priority.

Then the packetizability of the data objects can be decided by the following test:

- Packetizable if $\sum_{i=1}^{M} \frac{E_i}{T_i} \leq n \times (2^{1/n} - 1)$
- Not packetizable if $\sum_{i=1}^{M} \frac{E_i}{T_i} > 1$
- Inconclusive if $1 \geq \sum_{i=1}^{M} \frac{E_i}{T_i} > M \times (2^{1/M} - 1)$

If RMS is inconclusive, we must employ an exact test procedure that can be summarized as follows:

$$a_{i+1} = E_M + \sum_{j=1}^{M-1} \lceil \frac{a_i}{T_j} \rceil E_j, \ where \quad a_0 = \sum_{j=1}^{M} E_j.$$

The test terminates when $a_i > T_M$ (not packetizable) or when $a_i = a_{i+1} \leq T_M$ (packetizable).

To improve the packetizability of a set of objects that cannot be packetized by the first attempt, one can divide a data object into sub-objects to improve the packetizability. This technique is referred to as "period transformation." Ideally, if one divides a K bits object into K subobjects (each subobject has only one bit) one can enjoy the optimal packetizability and achieve the optimal channel utilization. However, the header overhead of a subprogram that transmits a subobject prevents us from dividing an object too finely. (For instance, if the header takes up four bytes, the overhead of transmitting an one-bit object is horrendous.) Thus, care must be taken when dividing a data object into subobjects for transmission.

We use an example to illustrate how the Carousel scheme works. Suppose we have four data objects to be broadcast in four programs and their broadcast periods T_i and their transmission times E_i are shown in Table 1.

Data i	E_i (s)	T_i (s)	E_i/T_i
D_1	5	50	1/10
D_2	10	70	1/7
D_3	40	120	1/3
D_4	80	200	2/5

Table 1: Carousel Example.

The RMS test shows that we can packetize the first three programs since $\sum_{i=1}^{3} E_i/T_i = 0.576 < 3(2^{1/3} - 1)$. The RMS test, however, cannot determine if we can packetize all four programs since $1 > \sum_{i=1}^{4} E_i/T_i = 0.976 > 4(2^{1/4} - 1)$. We follow the exact test to check if all four programs can be transmitted to meet their deadlines. Following the steps of the exact test, the answer we get is no. Finally, we perform period transformation to convert D_4 into ten subprograms as shown in Table 2. As a result, we can packetize all programs to meet their transmission deadlines.

Data i	E_i (s)	T_i (s)	E_i/T_i
D_4	8	20	2/5
D_1	5	50	1/10
D_2	10	70	1/7
D_3	40	120	1/3

Table 2: After Period Transformation.

So far, we assume that a data object is broadcast periodically. If slack bandwidth exists, one can use the slack to schedule aperiodic tasks or to schedule addition broadcast for the data objects that have been updated before their transmission schedule.

3.2.2.2 The Pyramid Scheme

The Carousel scheme can suffer from long initial latency. For example, if a program is run once an hour, the worst-case initial latency for watching the program is one hour. To remedy this long-delay problem, we can employ the Pyramid scheme for multicast proposed by [11] to packetize data.

The idea of the Pyramid scheme is to divide a program into subprograms, whose sizes are in a geometrical series. If the first subprogram is K bytes, the ith subprogram is $2^i \times K$ bytes, and the total size amounts to S_i. The Pyramid scheme then broadcasts each subprogram repeatedly. Figure 4 shows an example where a program is divided into four subprograms. The x-axis of the figure is time. The first subprogram lasts for one unit of time, the second for two, the third lasts for four, and the last for eight. Each subprogram is repeatedly transmitted. The receiver receives all subprograms at the same time. The worst-case initial delay is the duration of the first subprogram.

Let us see how the scheme works via an example execution. Suppose a user starts downloading the data object at the time indicated by the A arrow in the figure. The receiver at the client side starts receiving all four subprograms at A. After the first subprogram is completely received, the receiver start the playback (indicated in the figure by the B arrow). When the first subprogram is consumed, the second subprogram will have completely arrived at the receiver and the playback resumes without a hiccup. This process continues until the entire program ends.

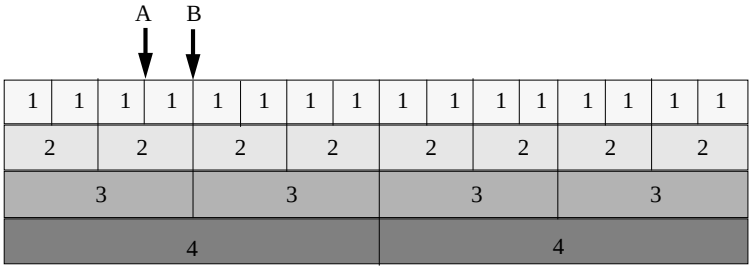


Figure 4: A Pyramid Examples.

The Pyramid scheme can enjoy very short initial latency if one is willing to trade bandwidth for performance. For a one-hour movie, we can divide the program into six subprograms that last for roughly $1, 2, 4, 8, 16$ and 32 minutes and transmit them repeatedly in six subchannels. In this case, we have a worst-case delay of only one minute. If we use the Carousel scheme to achieve the same one-minute latency, we have to broadcast the movie sixty times in an hour, one minute apart, which takes up sixty subchannels. The Pyramid scheme is clearly more desirable. In general, the Pyramid scheme can be a big win when a data object has a large N_i and a large S_i.

Note that for the Pyramid scheme to work, the receiver must be able to receive multiple subprograms simultaneously. We thus need an intelligent receiver, which can buffer data properly and efficiently at minimum cost. Below, we discuss the design of such a receiver.

3.3 Data Receivers

Designing a client/server dual system to support interactivity is a non-trivial task. On the one hand, the receiver must write broadcast signals to its disk to minimize memory use for staging streams. On the other hand, the receiver must read data from the disk into main memory before the decoder runs out of data. In addition, when simultaneous fast-scans (i.e., fast-forward and replay) are requested, the reads can happen at very high rates. The system must ensure that all IOs can be done not only on time to meet their deadlines but also at the minimum cost.

To further complicate matters, the design of the servers and clients faces at least three challenges:

- Not all data are equal. Data objects can be continuous, textual, binary, and so forth. (We have discuss data characterization in Section 3.1.)

- Not all data receivers are equal. Different receivers may have different available hardware resources and network bandwidth. If the performance of a receiver must degrade because of limited resources, it must degrade in a graceful manner.
- Not all subscribers are equal. Different viewers may have different viewing preferences. To measure a viewer's satisfaction, we must allow the user to specify what is important to the user and what is not. We thus assign to each data object a QoS ("Quality of Service") parameter. A viewer can assign a QoS value to each data object either implicitly or explicitly.

In this section we discuss our preliminary plans for tackling these challenges in three areas: resource management, data placement, and QoS maximization.

3.3.1 Resource Management

A receiver is equipped with a large and inexpensive disk for caching. The economical caching together with the random access capability of the disk enables a multimedia receiver to buffer a large amount of data to support time-shift operations. For AV data, a viewer can pause a live program to take a break from viewing while the broadcast stream continues arriving and being written to the local disk. The viewer can resume watching the program after the pause with a delay. He/She can also replay or fast-forward the program to get back in sync with the broadcast stream. For non-AV broadcast data, the receiver can stage the data on the disk, parse the data and then retain the data that are of interest (e.g., retain the score of a baseball game, the local weather, or the quotes of selected stocks).

Extending the DRAM cache with a large disk adds challenges to the receiver's buffer management. To ensure that IOs are efficiently scheduled, the receiver needs a *buffer manager* that can schedule the receiver's memory and disk bandwidth effectively to service a wide variety of interactive scenarios.

3.3.1.1 Buffer Management

We plan to use a buffer-management scheme [2] called "Four-page Ping-pong Scheme." This scheme is designed for AV data that have continuous constraints. However, since non-AV data may have to synchronize their playback time with AV data, they may be treated as continuous data with more liberal deadlines. For non-AV data that do not have playback deadline (e.g., software), we need just two pages for receiving data. In the remainder of this section, we assume that data are AV or related to a AV stream.

For the regular playback of a continuous data stream, the resource manager allocates four memory pages of equal size. These four pages are *ping-ponged* between the receiver and the processor (a decoder or an application may run on the processor) and are used to receive incoming data from the network, write data to the disk, read data from the disk, and provide data for decoding. As we will show shortly, the receiver and the processor each "owns" at most two pages at any time. The following example illustrates how the resource manager manages these four pages for a video playback.

At the start of the video playback, the resource manager allocates four pages, denoted by B_1, B_2, B_3 and B_4 (shown in Figure 5), for a stream. Once data begins arriving, the resource manager receives the data in one of the pages, say B_1. When B_1 is full, B_1 is available for decoding and hence the resource manager hands B_1 to the processor.[1] At the same time, the resource manager assigns another memory page, say B_2, to the receiver to continue receiving data. When page B_2 is full, the resource manager hands B_2 to the processor and assigns B_3 to the receiver to continue receiving data. At this time, the processor "owns" pages B_1 and B_2, and the receiver B_3.

[1] In this example, we assume that the playback starts once B_1 is filled up and is assigned to the processor. The playback can start earlier or later, depending on the difference between the data delivery and playback rates. For example, if the data delivery rate is substantially lower than the playback rate, the resource manager must accumulate enough data on disk before the playback can start to prevent display hiccups.

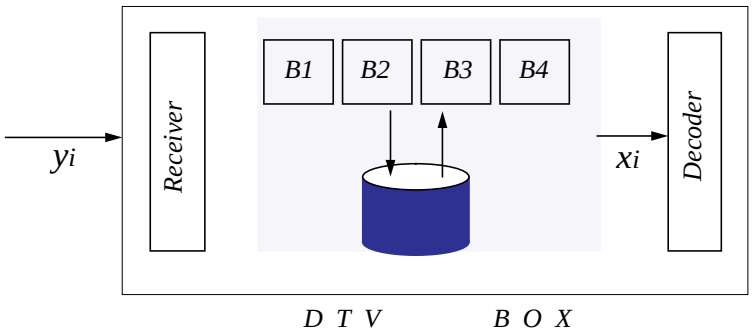


Figure 5: The Four-page Ping-pong Scheme.

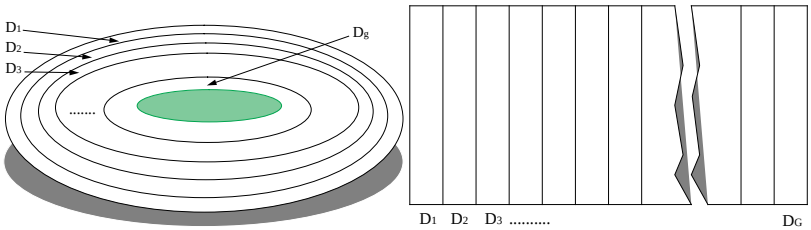
One of two events occurs next: either the data in page B_1 is used up by the processor or page B_3 is filled up by the incoming packets. The resource manager takes different actions according to which event occurs first.

- If B_1 is used up first, the resource manager simply returns the page to the free pool.
- If B_3 is filled up first, the resource manager writes the page to the disk. Note that the processor does not need B_3 immediately. The processor needs only B_2 to safeguard a hiccup-free playback. When B_3 is being written to the disk, B_1 and B_2 are held by the processor. The resource manager must use page B_4 to receive incoming data. B_4 must be large enough for the disk to complete writing B_3 to disk, or else the receiver runs out of memory space to receive data once B_4 is full.

At this time, one of two things can happen next: the data in page B_1 is used up by the processor or page B_4 is filled up by the arriving data.

- When B_1 has been consumed by the processor, the resource manager starts consuming B_2. At the same time, the resource manager has to make sure that the data after that in B_2 is made available to the processor before the processor uses up B_2, or hiccups occur. If the processor uses up B_1 before B_3 is full, the resource manager assigns B_3 to the processor as soon as the page is filled up. If B_3 is filled up before page B_1 is consumed, an IO to write B_3 to the disk may be in progress or may have been completed. In the former case, the resource manager still makes B_3 available to the processor by ignoring the write (canceling the write may not be possible). If the write has been completed, the resource manager reads the data back from the disk into B_1 (the processor uses the data in B_1 after B_2 is consumed). Note that the size of page B_2 must be large enough to allow enough time for page B_1 to be replenished.
- If page B_4 is filled up first, the resource manager writes the data to the local disk. Since either page B_1 or B_3 must have been freed at this time, the resource manager uses the available page as the receiving buffer.

In summary, the resource manager is driven by two events: *page-consumed*, when the processor has consumed a page, and *page-full*, when the receiver has filled up a page. When the *page-consumed* event occurs, the resource manager may need to read a page of data from the disk before the processor uses up the next page. The page thus must be large enough to provide data to the processor before the read is completed. When the *page-full* event occurs, the resource manager may need to write the page to the disk to free up space for use. The page must be large enough so that during the write the receiver cannot fill up another page. This four-page ping-pong scheme has three benefits:



(a) Disk Partitions (b) Disk Matrix

Figure 6: Disk Partitions

1. It serves as a rubber band to cushion the difference between the incoming and decoding bitrates.
2. It does not perform unnecessary memory-to-memory or memory-to-disk copying if the data is played back in real-time.
3. If the delay is desirable (a viewer pauses the playback), it uses the disk instead of memory to buffer data and limits the memory requirement to four pages. This ping-pong scheme minimizes memory requirement and hence caching cost for supporting regular playback and pause.

3.3.1.2 Supporting Fast-Scans

We call fast-forward and replay operations *fast-scans*. For both AV and conventional textual data, fast-scans are possible. For AV data, a fast-scan plays back a media stream at K times its encoding rate (e.g., $K = 4, 8, and\ 16$) in either a forward direction (fast-forward) or a backward direction (replay). To support a K times fast-scan, the receiver may have to supply data to the processor at K times the regular rate.[2] The processor then decodes and displays one out of every K frames. For processing conventional data, the receiver may want to speed up IOs K times even the data do not synchronize with AV data (e.g., during skimming a powerpoint presentation). Since the data are cached on the local disk, the receiver must perform disk reads at a K-time rate and needs memory to stage these data. This additional memory can be huge if IO resolution is low or disk latency is high. We will investigate novel data placement and disk scheduling policies that can improve IO resolution and thereby reduces the DRAM requirement for supporting fast-scans.

3.3.2 Data Placement

Minimizing disk-arm movement provides two benefits. First, it reduces disk latency and may reduce the memory requirement for staging continuous data. Second, being able to retrieve data fast from disk reduces the hiccup time. One can reduce disk latency by employing a disk partitioning scheme [10]. To shorten the distance that the disk-arm must travel, one can restrict the disk-arm movement for each IO to a smaller distance. Figure 6(a) shows that a disk is partitioned into G concentric cylinder groups, each of which occupies an average of $\frac{CYL}{G}$ cylinders. Figure 6(b) shows the cylinder groups presented in a matrix form, where each column represents a cylinder group. If we place the media data of a DTV program in one cylinder group, we can bound the worst-case seek distance by CYL/G cylinders. For instance, we can divide

[2] If the receiver does not parse the stream to learn where the offset of each frame is located, the receiver must read in the data sequentially, parse meta-data, then skip to the desired frame. Alternatively, the receiver can pre-parse the stream before storing the data on the disk, and build an index table for fast-scans. This pre-parsing may allow the receiver to avoid reading all blocks during a fast-scan operation. However, processing conventional data can also require fast IOs, where data blocks cannot be skipped.

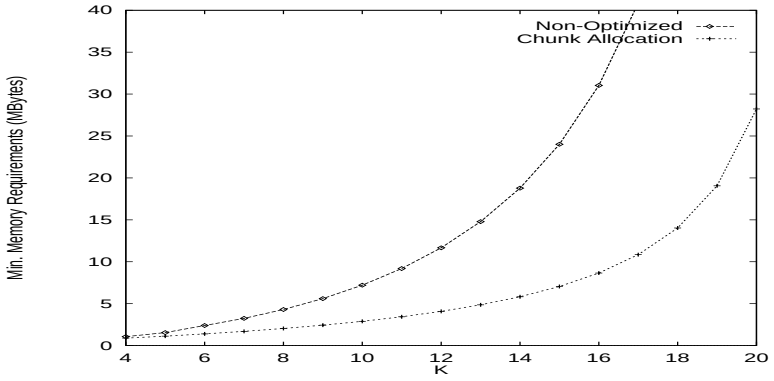


Figure 7: Memory Requirement vs. K

a 25-GByte hard disk into 15 cylinder groups, each of which can hold a DTV program of up to 30 minutes. The worst-case seek distance is reduced to 7% of the disk radius.

Figure 7 presents our preliminary experimental result on memory requirement using a Quantum Fireball disk. The x-axis of the figure indicates the speedup of the fast-scan and the y-axis shows the minimum memory required to support the K-time fast-scan. The upper curve in the figure shows the memory requirement. This curve shows that the memory requirement of the non-optimized scheme that we have presented grows without bound when K approaches 16. Employing the disk partitioning scheme mentioned above significantly reduces the memory requirement as indicated in the lower curve in the figure. For instance, when $K = 16$, the DRAM requirement is less than 10 MBytes. This low memory cost makes the receiver affordable, or put it the other way, given a fixed amount of memory, the receiver can stage more data in its main memory for performance gain.

3.3.3 Maximizing QoS

Given some data objects with their characteristics, the receiver's resources may not be adequate to support a particular interactive scenario or a data processing request that requires multiple data objects. Therefore, it is critical for the resource manager to be adaptive to the resource constraints and to degrade, if degradation is unavoidable, in a graceful manner. Given limited memory and disk bandwidth, the design objective of a receiver is to prioritize resource allocation in order to maximize the user's satisfaction.

To measure a user's satisfaction, we must allow the user to have a say about what is important to the user and what is not. We thus associate each data object with a *QoS parameter*. A viewer can assign a QoS value to each data object, either implicitly or explicitly. For instance, if a user decides to turn off all interactive features, the resource manager can assign a QoS factor of zero to data objects that are needed to support interactive features. Regarding the data objects that are needed to support the playback, a higher QoS can be assigned to mission-critical data objects, such as broadcast video and audio streams and a lower QoS can be assigned to optional data objects such as captions and applets.

Given N_{all} requested data objects, the goal of the resource manager is to schedule N data objects ($N \leq N_{all}$) to maximize the total QoS under the resource constraint. We use α_i to denote the QoS requirement assigned to the ith data object ($0 \leq \alpha_i \leq 1$). Let j represent the scheduling order for N data objects. We would like to schedule the service to these N data objects in an order that maximizes the sum of

the QoS. The objective function can be written as

$$Max \sum_{j=1}^{N} \alpha_j \times f_j(t - \sum_{k=1}^{j} \tau_k), \tag{10}$$

which is subject to the memory constraint

$$\sum_{j=1}^{N} S_j \leq M_{avail}.$$

Scheduling periodic tasks to meet their deadlines have been extensively studied in the real-time community []. However, the scheduling problem that a multimedia receiver faces is more complex than that of traditional real-time systems for at least three reasons:

1. Variable execution time. Because of the variable bitrate (VBR) encoding scheme used by video data, the required CPU execution time can vary from decoding one frame to another.

2. Just-in-time scheduling. Meeting the deadline of a task too early may use up additional resources and can consequently introduce bottlenecks in the system. For instance, if one decodes frames at a faster rate than the rate that the frames are consumed, the receiver must allocate more memory to stage the decoded frames. This additional buffer requirement may hinder other processes from getting their needed memory.

3. Unpredictable IO interrupts. An IO completion that incurs an interrupt can cause the kernel to switch context from the current critical job (e.g., decoding a frame) to handle this interrupt. An incoming email, an incoming cell phone call, a Web page download, etc. can all cause the running process to miss its deadline. Traditional real-time CPU scheduling schemes have largely ignored IO issues when dealing with scheduling.

We will investigate CPU scheduling policies that can tackle above challenges.

4 Timetable and Milestones

Two Ph.D. students: Kingshy Gho and Raju Rangaswami have been assigned to this πDVCR project. I plan to recruit two more students once this proposal is approved to expedite the development of the technologies.

The following is a list of major milestones we expect to complete. Due to space limitations, the description below is brief. Also, note that for each milestone, there are others, not shown below, for the associated experimental evaluations, report generation and technology transfer.

Tasks Completed By Year 1:

- Data placement: Complete the design of data placement polices that support multi-stream fast-scans with a low DRAM requirement.
- Real-time IO scheduling: Complete the design of a real-time IO scheduler that can schedule multiple simultaneous IO requests to meet their deadlines and at the same time achieve low latency for switching channels.
- Complete the development of a cost model that produces the optimal broadcast schedule for data services.
- Experiment with NIST's standards to produce a prototype that support content interactivity.

Tasks Completed By Year 2:

- CPU and IO integrated real-time scheduling: Complete the design of a real-time CPU and IO scheduler. This task may involve fundamental computer architecture redesign.
- Data packetization: Complete the design of an efficient data packetization scheme for data services.
- Indexing structures: Complete the design of spatial and temporal indexing structures that assist efficient channel surfing and effective interactive operations.
- Implement the second piDVCR prototype that integrates the above features.

5 Conclusion

In this proposal, we discussed the potential and importance of data broadcasting as a data delivery mechanism. We showed why the database community may be interested in extending its research into this area. We presented challenges that data broadcasting and interactive TV face and discussed some of our preliminary ideas. In particular, we focused our discussion on three areas: how to characterize different data objects, how a server schedule its broadcast and how a receiver manages its resources intelligently. Data broadcasting and interactive TV are still in their infancy. We believe that much work will be necessary to make this data transport an effective one and our πDVCR project will produce exciting technologies for advancing the field.

References

[1] http://gartner4.gartnerweb.com/dq/static/about/press/pr-b200008.html. Feb. 2000.

[2] E. Chang. Maximizing qos for interactive dtv clients. *The Computer Communications Journal (Special Issue), Elsevier*, 23(3):205–218, Febeuary 2000.

[3] E. Chang and H. Garcia-Molina. Bubbleup - Low latency fast-scan for media servers. *Proceedings of the 5th ACM Multimedia Conference*, pages 87–98, November 1997.

[4] E. Chang and H. Garcia-Molina. Effective memory use in a media server. *Proceedings of the 23rd VLDB Conference*, pages 496–505, August 1997.

[5] E. Chang and H. Garcia-Molina. Medic: Memory and disk cache for multimedia clients. *Stanford Technical Report SIDL-WP-1997-0076 (Extended Version)*, October 1997.

[6] E. Chang and H. Garcia-Molina. Medic: Memory and disk cache for multimedia clients. *Proceedings of IEEE International Conference on Multimedia Computing and Systems*, pages 493–499, June 1999.

[7] E. Chang, H. Garcia-Molina, and C. Li. 2d Bubbleup - Managing parallel disks for media servers (extended version). *Stanford Technical Report SIDL-WP-1998-0090*, February 1998.

[8] M. S. Chen and D. D. Kandlur. Downloading and stream conversion: Supporting interactive playout of videos in a client station. *ICMCS*, May 1995.

[9] R. Franco. Hype on demand. *http://www.redherring.com/industries/2000/0801/ind-video080100.html*, August 2000.

[10] S. Ghandeharizadeh, S. Kim, and C. Shahabi. On configuring a single disk continuous media server. *Sigmetrics Performance Evaluation*, 23(1):37–46, May 1995.

[11] K. Hua and S. Sheu. Skyscraper broadcasting: A new broadcasting scheme for metropolitan video-on-demand systems. *ACM Sigcomm*, pages 89–101, Sept. 1997.

[12] M. Klein, T. Ralya, B. Pollak, and R. Obenza. *A Practitioner's Handbook for Real-Time Analysis: Guide to Rate-Monotonic Analysis for Real-Time Systems*. Kluwer Academic Publishers, 1993.

[13] L. Sha, R. Rajkumar, and S. Sathaye. Generalized rate monotonic scheduling theory, a framework of developing real-time systems. *Proceedings of The IEEE*, January 1994.

[14] P. Shenoy and H. Vin. Efficient support for interactive operations in multi-resolution video servers. *ACM Multimedia Systems*, 7(3), May 1999.